API REST com Spring Boot, JPA e MySQL no VS Code

Oct 20, 2026 · @Vânio Aníbal Macamo

Este guia leva-vos de um projecto vazio a uma API REST com CRUD completo de `Cliente`, ligada a MySQL através de JPA, executada no VS Code e testada no Postman.

1. Pré-requisitos

Instalem e confirmem estas ferramentas antes de começar:

Ferramenta	Versão	Como confirmar
JDK	17 ou superior	`java -version`
Visual Studio Code	Recente	Abrir o programa
MySQL Server	8 ou superior	`mysql --version`
Postman	Recente	Abrir o programa

O Maven não precisa de instalação separada: o projecto gerado inclui o wrapper `mvnw` (Windows: `mvnw.cmd`).

2. Extensões do VS Code

Sem estas extensões o VS Code não reconhece projectos Spring, não mostra o botão Run e não completa propriedades. Abram o separador **Extensions** (`Ctrl+Shift+X`) e instalem:

1. **Extension Pack for Java** (Microsoft)
2. **Spring Boot Extension Pack** (VMware / Broadcom)
3. **Thunder Client** ou **REST Client** (opcional, alternativa ao Postman dentro do VS Code)

3. Criar o projecto no start.spring.io

O projecto gera-se em start.spring.io com os campos abaixo.

Campo	Valor
Project	Maven
Language	Java
Spring Boot	Versão estável mais recente (sem SNAPSHOT, M1 ou RC no nome)
Group	`mz.co.iscim`
Artifact	`aula-spring`
Packaging	Jar
Java	17 ou superior (igual ao JDK instalado)

Clicar em **ADD DEPENDENCIES** (`Ctrl+B`) e adicionar:

- **Spring Web**
- **Spring Data JPA**
- **MySQL Driver**
- **Validation**
- **Spring Boot DevTools** (opcional, reinicia a aplicação ao guardar)

Clicar em **GENERATE** (`Ctrl+Enter`) para descarregar o ficheiro `.zip` e extraí-lo para a pasta de trabalho.

4. Abrir o projecto no VS Code

1. **File → Open Folder** e escolher a pasta que contém o `pom.xml` (não a pasta de cima).
2. Se o VS Code perguntar, clicar em **Yes, I trust the authors**.
3. Esperar que o VS Code descarregue as dependências e indexe o projecto (progresso no canto inferior direito). Só depois disso os erros vermelhos desaparecem.

5. Base de dados e configuração

A aplicação só arranca se a base de dados existir e as credenciais estiverem correctas.

Passo 1 — criar a base de dados (no MySQL Workbench ou no terminal `mysql -u root -p`):

```
CREATE DATABASE spring_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

Passo 2 — configurar `src/main/resources/application.properties`:

```
spring.application.name=aula-spring

spring.datasource.url=jdbc:mysql://localhost:3306/spring_db?
useSSL=false&serverTimezone=Africa/Maputo&allowPublicKeyRetrieval=true
spring.datasource.username=root
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

server.port=8080
```

Se o `root` tiver password, escrevê-la depois do `=`, sem espaços no fim da linha. Se não tiver, deixar vazio como acima.

`ddl-auto=update` cria e actualiza as tabelas automaticamente a partir das entidades. Serve para aulas e desenvolvimento; em produção usa-se `validate` ou `none`.

6. Estrutura de pacotes

O pedido percorre quatro camadas: **Postman → Controller → Service → Repository → MySQL**. Cada camada vive num pacote próprio, criado dentro do pacote da classe `@SpringBootApplication`:

```
mz.co.iscim.aula_spring
├── AulaSpringApplication.java
├── model/        → Cliente.java
├── repository/   → ClienteRepository.java
├── service/      → ClienteService.java
├── controller/   → ClienteController.java
└── exception/    → RecursoNaoEncontradoException.java,
GlobalExceptionHandler.java
```

Os pacotes têm de ficar **dentro** do pacote da classe principal, senão o Spring não detecta as classes.

7. Código

Criar cada ficheiro no pacote indicado. No Spring Boot 3 e 4 os imports de persistência e validação começam por `jakarta`, não `javax`.

7.1 Entidade — `model/Cliente.java`

```
package mz.co.iscim.aula_spring.model;

import jakarta.persistence.*;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;

@Entity
@Table(name = "clientes")
public class Cliente {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "O nome é obrigatório")
    @Column(nullable = false)
    private String nome;

    @Email(message = "Email inválido")
    private String email;

    private String telefone;

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public String getTelefone() { return telefone; }
    public void setTelefone(String telefone) { this.telefone = telefone; }
}
```

7.2 Repositório — `repository/ClienteRepository.java`

```
package mz.co.iscim.aula_spring.repository;

import mz.co.iscim.aula_spring.model.Cliente;
import org.springframework.data.jpa.repository.JpaRepository;

public interface ClienteRepository extends JpaRepository<Cliente, Long> {
}
```

O `JpaRepository` já traz `findAll`, `findById`, `save`, `deleteById` e `existsById`, sem escrever SQL.

7.3 Excepção — `exception/RecursoNaoEncontradoException.java`

```
package mz.co.iscim.aula_spring.exception;

public class RecursoNaoEncontradoException extends RuntimeException {
    public RecursoNaoEncontradoException(String mensagem) {
        super(mensagem);
    }
}
```

7.4 Serviço — `service/ClienteService.java`

Contém a lógica de negócio e é a única camada que fala com o repositório.

```
package mz.co.iscim.aula_spring.service;

import mz.co.iscim.aula_spring.exception.RecursoNaoEncontradoException;
import mz.co.iscim.aula_spring.model.Cliente;
import mz.co.iscim.aula_spring.repository.ClienteRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

@Service
public class ClienteService {

    private final ClienteRepository repository;
```

```
    public ClienteService(ClienteRepository repository) {
        this.repository = repository;
    }

    public List<Cliente> listar() {
        return repository.findAll();
    }

    public Cliente buscar(Long id) {
        return repository.findById(id)
                .orElseThrow(() -> new RecursoNaoEncontradoException("Cliente
não encontrado: " + id));
    }

    @Transactional
    public Cliente criar(Cliente cliente) {
        cliente.setId(null); // garante que é um INSERT
        return repository.save(cliente);
    }

    @Transactional
    public Cliente actualizar(Long id, Cliente dados) {
        Cliente existente = buscar(id);
        existente.setNome(dados.getNome());
        existente.setEmail(dados.getEmail());
        existente.setTelefone(dados.getTelefone());
        return repository.save(existente);
    }

    @Transactional
    public void eliminar(Long id) {
        if (!repository.existsById(id)) {
            throw new RecursoNaoEncontradoException("Cliente não encontrado:
" + id);
        }
        repository.deleteById(id);
    }
}
```

7.5 Controller — `controller/ClienteController.java`

Expõe os endpoints HTTP e delega o trabalho ao serviço.

```
package mz.co.iscim.aula_spring.controller;

import jakarta.validation.Valid;
import mz.co.iscim.aula_spring.model.Cliente;
import mz.co.iscim.aula_spring.service.ClienteService;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/clientes")
public class ClienteController {

    private final ClienteService service;

    public ClienteController(ClienteService service) {
        this.service = service;
    }

    @GetMapping
    public List<Cliente> listar() {
        return service.listar();
    }

    @GetMapping("/{id}")
    public Cliente buscar(@PathVariable Long id) {
        return service.buscar(id);
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Cliente criar(@Valid @RequestBody Cliente cliente) {
        return service.criar(cliente);
    }

    @PutMapping("/{id}")
    public Cliente actualizar(@PathVariable Long id, @Valid @RequestBody
Cliente cliente) {
        return service.actualizar(id, cliente);
    }

    @DeleteMapping("/{id}")
```

```
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void eliminar(@PathVariable Long id) {
        service.eliminar(id);
    }
}
```

7.6 Tratamento de erros — `exception/GlobalExceptionHandler.java`

Sem esta classe, um cliente inexistente devolve erro 500 em vez de 404.

```
package mz.co.iscim.aula_spring.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(RecursoNaoEncontradoException.class)
    public ResponseEntity<Map<String, String>>
naoEncontrado(RecursoNaoEncontradoException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
                .body(Map.of("erro", ex.getMessage()));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, String>>
validacao(MethodArgumentNotValidException ex) {
        Map<String, String> erros = new HashMap<>();
        ex.getBindingResult().getFieldErrors()
                .forEach(e -> erros.put(e.getField(),
e.getDefaultMessage()));
        return ResponseEntity.badRequest().body(erros);
    }
}
```

8. Executar a aplicação

Há três formas de arrancar; qualquer uma serve:

1. Abrir `AulaSpringApplication.java` e clicar em **Run** acima do método `main`.
2. Usar o painel **Spring Boot Dashboard** na barra lateral e clicar em play.
3. No terminal, na pasta do projecto: `.\mvnw.cmd spring-boot:run` (Linux/macOS: `./mvnw spring-boot:run`).

A aplicação arrancou bem quando o log mostra `Started AulaSpringApplication` e o Hibernate executa `create table clientes`. O servidor fica à escuta em `http://localhost:8080`.

9. Testar o CRUD no Postman

Preparação: em **Environments → New** criar a variável `baseUrl` com o valor `http://localhost:8080` e seleccionar esse ambiente no canto superior direito. Nos pedidos escreve-se `{{baseUrl}}/api/clientes`.

Nos pedidos POST e PUT, separador **Body → raw → JSON**, com o conteúdo:

```
{
  "nome": "Maria",
  "email": "maria@email.com",
  "telefone": "841234567"
}
```

Operação	Método	URL	Body	Resposta esperada
Criar	POST	`{{baseUrl}}/api/clientes`	JSON completo	201 Created, com `id`
Listar todos	GET	`{{baseUrl}}/api/clientes`	—	200 OK, array JSON
Buscar um	GET	`{{baseUrl}}/api/clientes/1`	—	200 OK, ou 404
Actualizar	PUT	`{{baseUrl}}/api/clientes/1`	JSON completo	200 OK, ou 404

Operação	Método	URL	Body	Resposta esperada
Eliminar	DELETE	`{{baseUrl}}/api/clientes/1`	—	204 No Content, ou 404

O PUT substitui todos os campos, por isso enviem sempre o objecto completo.

Testar também os erros:

Teste	Como	Esperado
Nome vazio	POST com `"nome": ""`	400 com a mensagem do campo `nome`
Email inválido	POST com `"email": "abc"`	400 com "Email inválido"
Id inexistente	GET `/api/clientes/999`	404 com `{"erro": "Cliente não encontrado: 999"}`

Confirmar na base de dados:

```
USE spring_db;
SELECT * FROM clientes;
```

10. Problemas comuns

O erro mais frequente é de ligação ou autenticação no MySQL; a causa vê-se sempre na primeira linha `Caused by` do log.

Sintoma	Causa provável	Solução
`Access denied for user 'root'@'localhost'`	Password errada no `application.properties`	Testar no terminal com `mysql -u root -p` e copiar a password que funciona
`using password: YES` mas o `root` não tem password	A aplicação correu com configuração antiga	Guardar o ficheiro (`Ctrl+S`), parar a aplicação, correr `.\mvnw.cmd clean` e voltar a executar

Sintoma	Causa provável	Solução
`Communications link failure`	MySQL desligado ou porta errada	Arrancar o serviço MySQL e confirmar a porta 3306
`Unknown database 'spring_db'`	Base de dados não criada	Executar o `CREATE DATABASE` da secção 5
`Unable to determine Dialect`	Consequência de falhar a ligação à base de dados	Resolver primeiro o erro de ligação acima
`415 Unsupported Media Type`	Body do Postman não está em raw → JSON	Escolher **raw** e **JSON** no separador Body
`Could not get any response` no Postman	Aplicação parada ou porta diferente	Confirmar `Started AulaSpringApplication` no log e `server.port`
404 em todos os pedidos	URL errado ou controller fora do pacote principal	Confirmar `/api/clientes` e a localização do pacote `controller`
Erros vermelhos no código depois do import	Cache do Java Language Server	`Ctrl+Shift+P` → **Java: Clean Java Language Server Workspace**
Porta 8080 ocupada	Outro programa a usá-la	Mudar `server.port=8081` no `application.properties`

Alternativa ao `root`: criar um utilizador só para a aplicação.

```
CREATE USER 'spring_user'@'localhost' IDENTIFIED BY 'spring123';
GRANT ALL PRIVILEGES ON spring_db.* TO 'spring_user'@'localhost';
FLUSH PRIVILEGES;
```

Depois, em `application.properties`, usar `spring.datasource.username=spring_user` e `spring.datasource.password=spring123`.